



Sistemas Distribuidos

Profesor: Oscar Manuel Ruiz Hurtado

Práctica s05

Mexi Banco con Docker Compose

Sierra García Mariana

Grupo: 02

Facultad de Ingeniería

Evidencias de la práctica s05

1. Salud de la aplicación

Primero verificamos que el contenedor de la aplicación esté levantado y exponga correctamente sus servicios, se realiza una petición al *endpoint* de monitoreo.

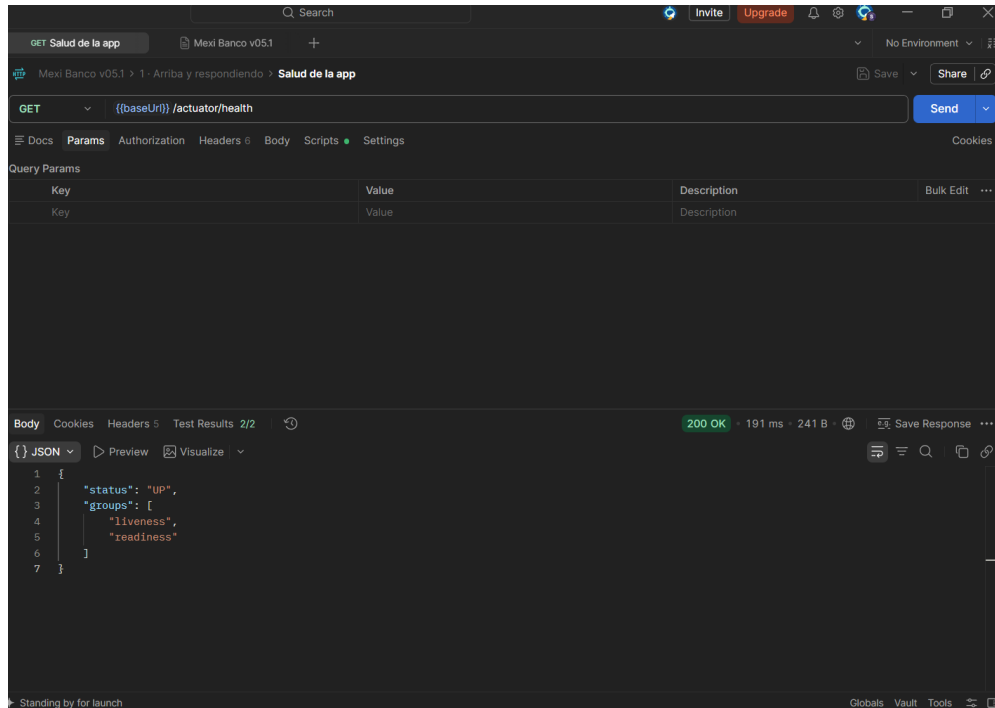


Figura 1: Respuesta 200 OK del endpoint /actuator/health.

2. Creación y consulta de cuentas

Se inicializan los datos y se crean dos cuentas en el sistema (origen y destino) y se verifica que los saldos iniciales definidos en el cuerpo de la petición se hayan guardado correctamente en la base de datos

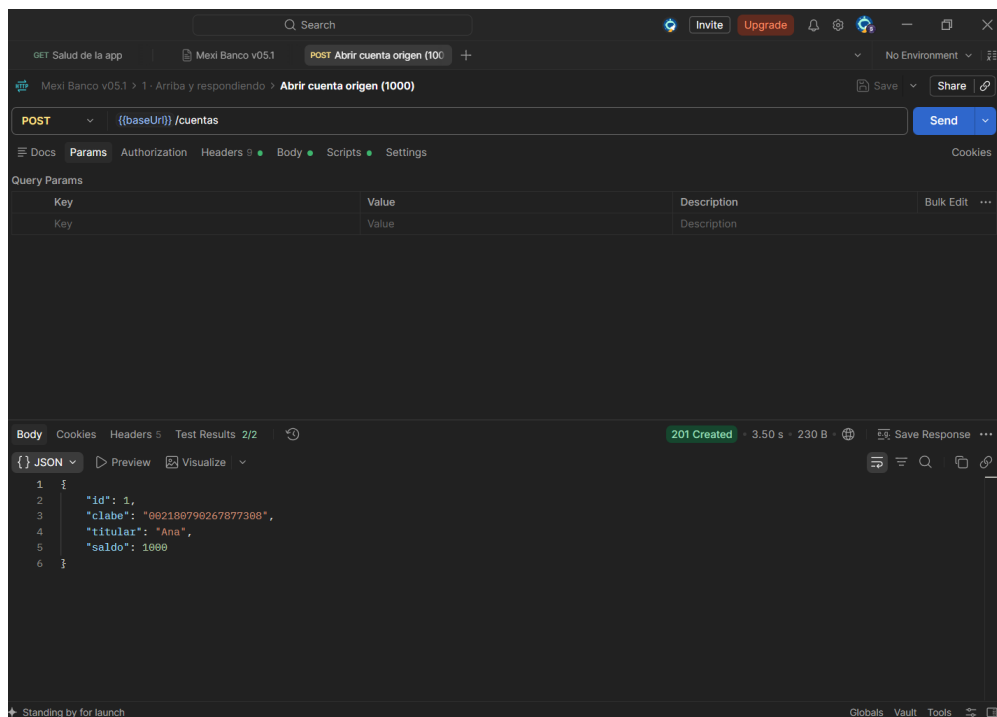


Figura 2: Creación exitosa de la cuenta origen con \$1000.

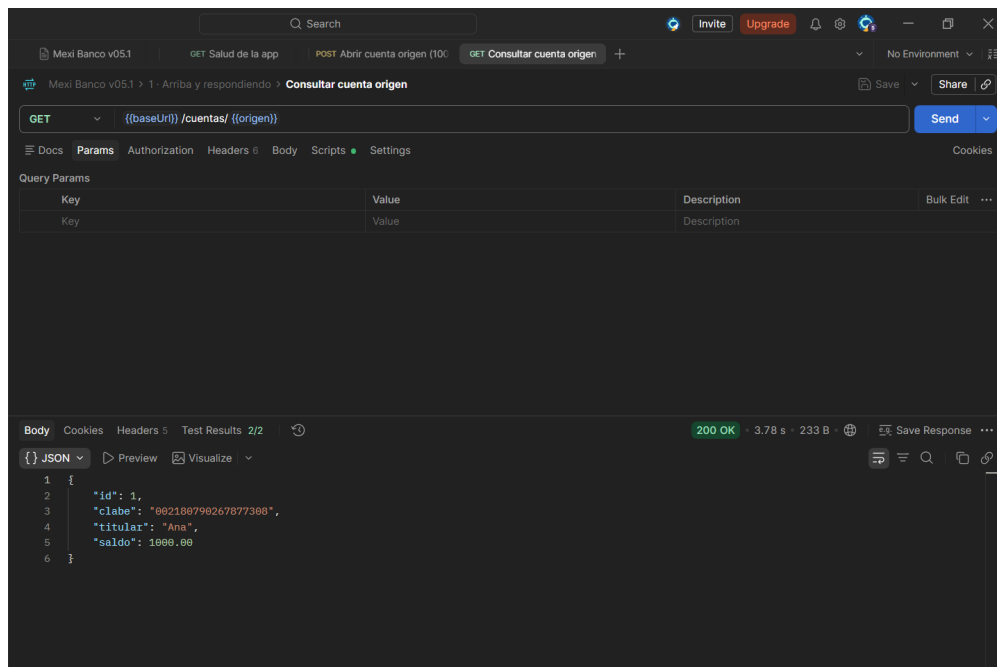


Figura 3: Consulta de la cuenta origen, confirmando su existencia

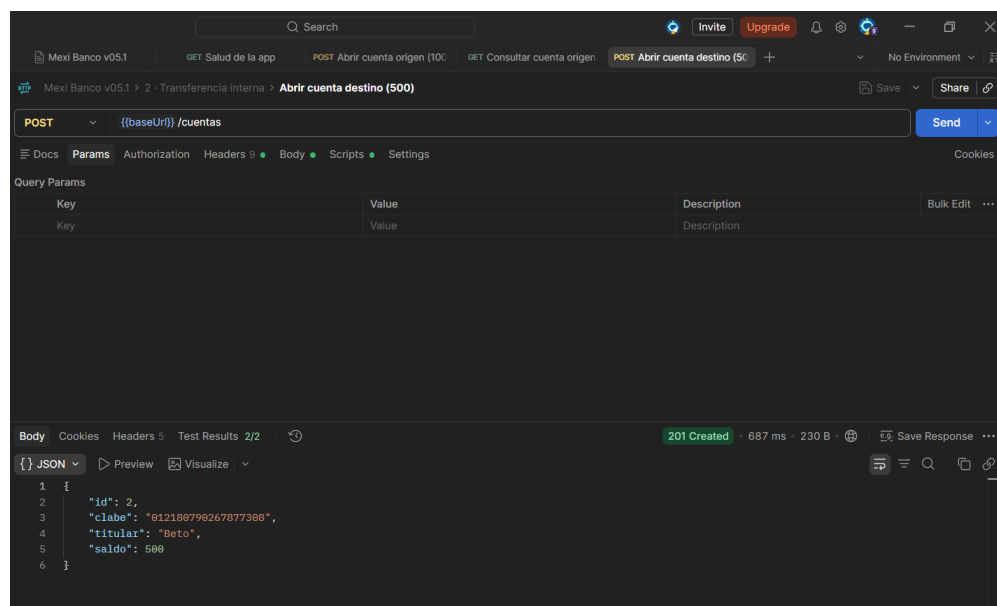


Figura 4: Creación de la cuenta destino con \$500

3. Transferencias internas

Se prueba el flujo básico de transferencia entre las dos cuentas creadas. despues de que se realiza la operacion es necesario consultar ambas cuentas para garantizar que se aplicó el principio de partida doble que es el origen debe tener \$200 menos y el destino \$200 más.

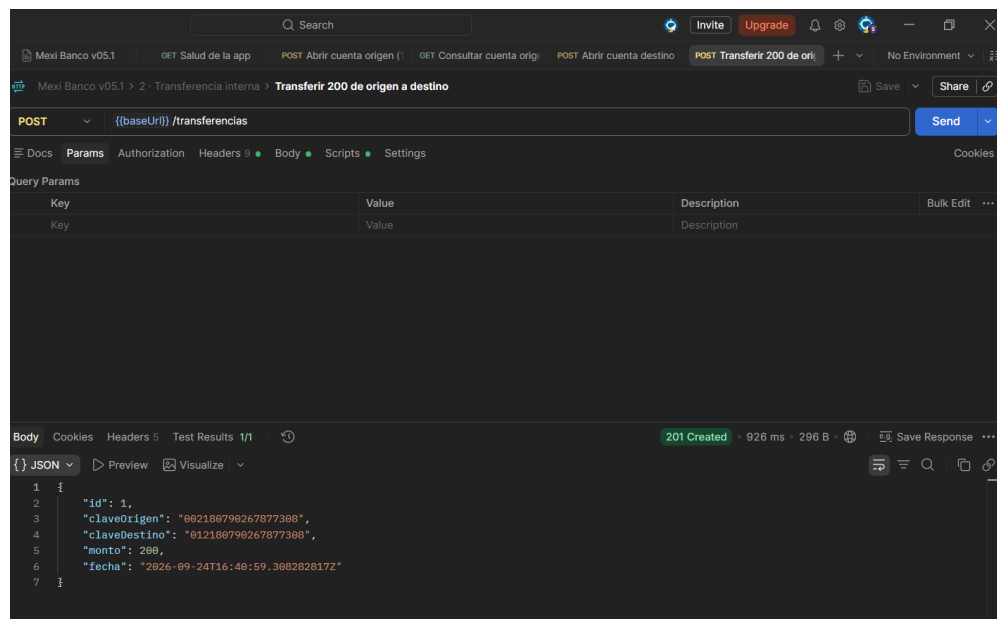


Figura 5: Petición para transferir \$200 del origen al destino.

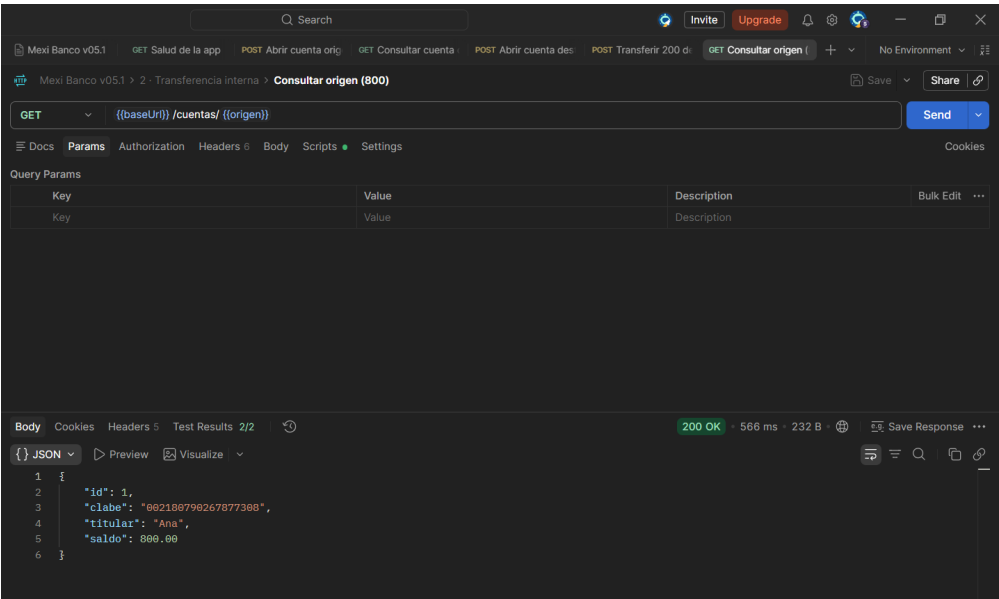


Figura 6: El saldo del origen se reduce correctamente a \$800.

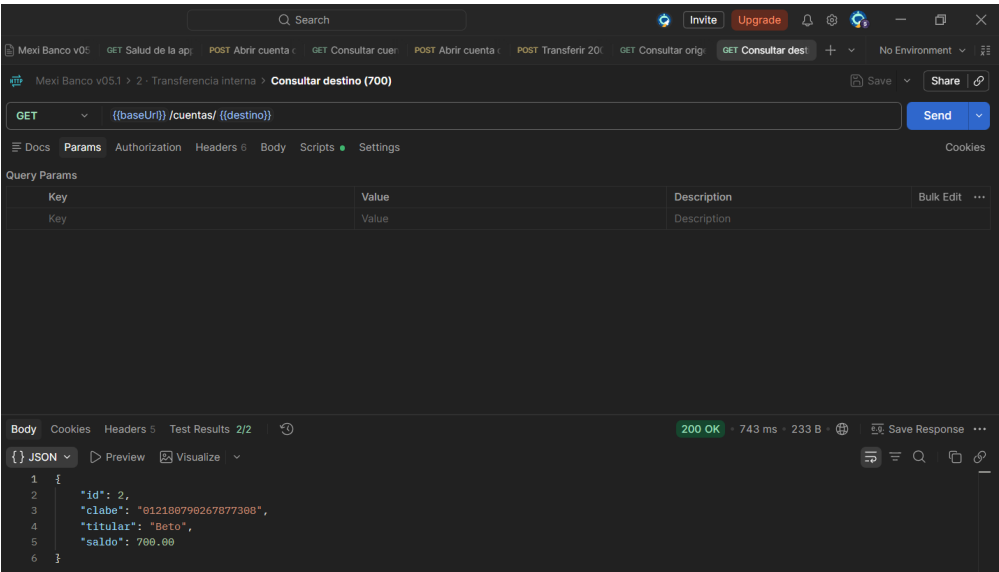


Figura 7: El saldo del destino aumenta correctamente a \$700

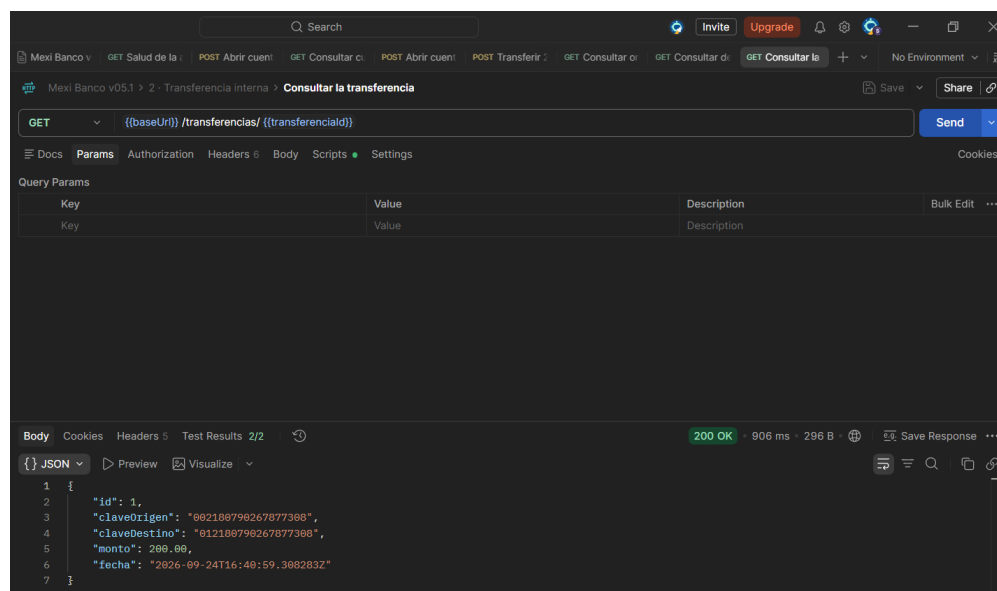


Figura 8: Registro exitoso de la transferencia en el sistema.

4. Operaciones SPEI e Idempotencia

Es importante evitar cobros duplicados si hay un error de red y por esto se prueba el envío de un SPEI usando un **Idempotency-Key**. Al repetir la petición exacta el servidor debe responder de forma exitosa sin aplicar el cargo dos veces

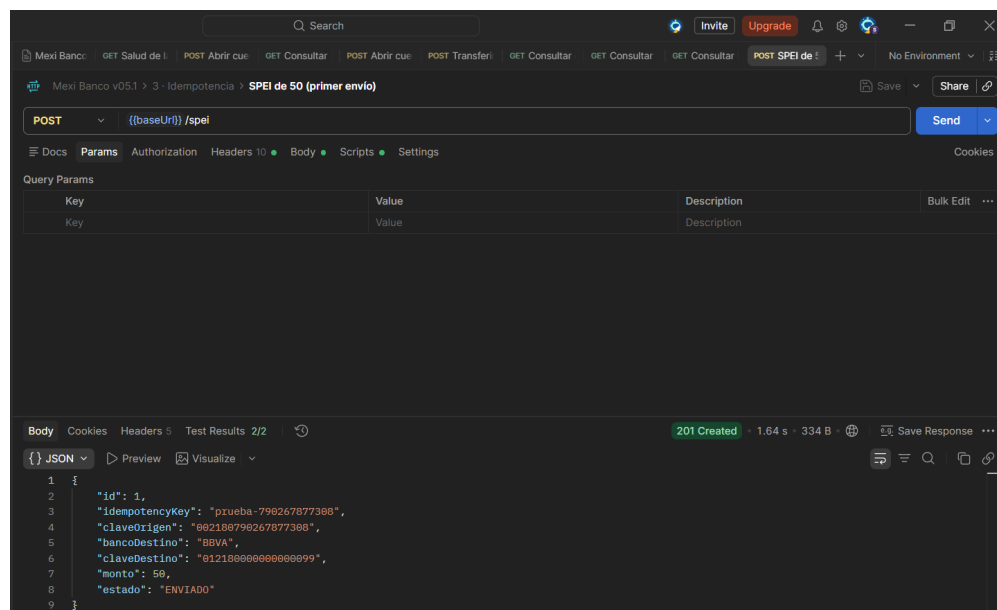


Figura 9: Primer envío de SPEI por \$50.

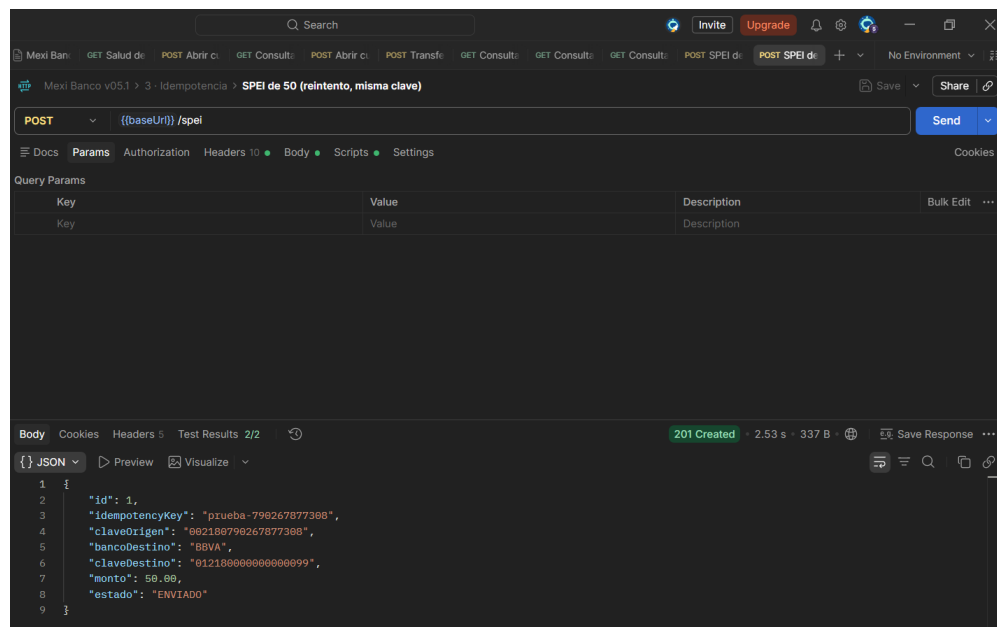


Figura 10: Reintento del mismo SPEI usando la misma clave de idempotencia

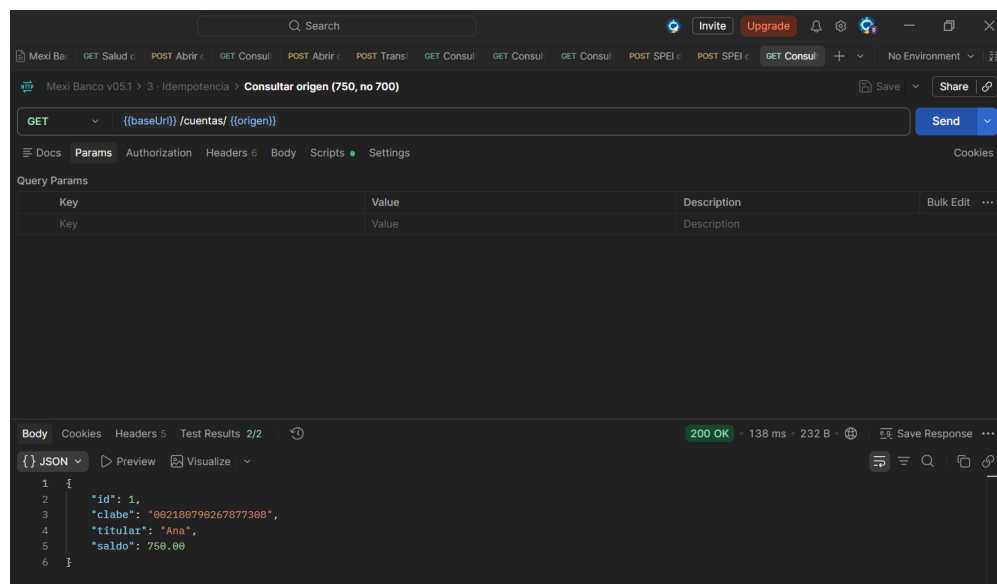


Figura 11: El saldo refleja \$750, comprobando que el SPEI se cobró solo una vez.

5. Estados de Cuenta y Notificaciones

El sistema debe mantener un registro inmutable de todos los movimientos y generar avisos, posteriormente se verifican estos endpoints para asegurar la trazabilidad de la cuenta origen.

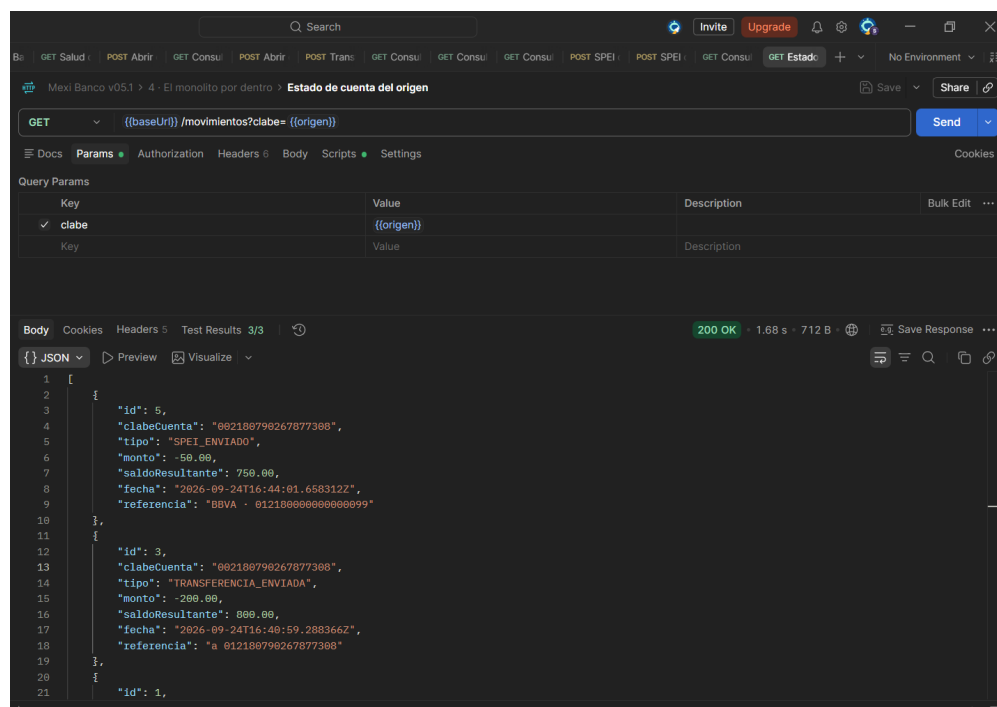


Figura 12: Estado de cuenta con todos los movimientos generados.

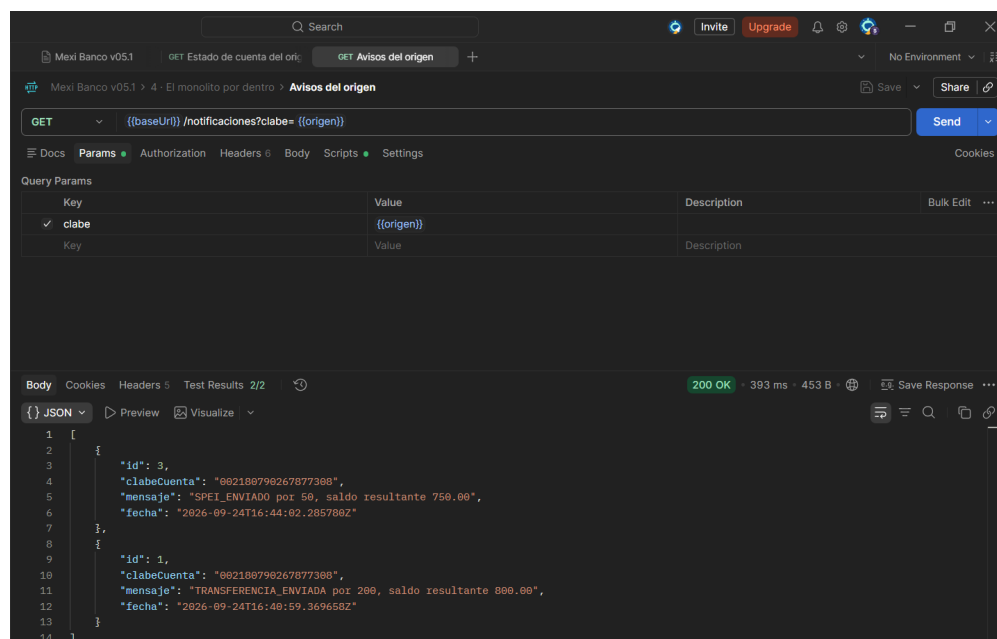


Figura 13: Notificaciones y avisos registrados para la cuenta.

6. Casos de Error y Reglas de Negocio

Una API robusta debe rechazar peticiones mal formadas o que violen reglas de negocio, se muestra cómo el sistema devuelve códigos de error HTTP apropiados (400, 404, 409, 422) y se comprueba que las operaciones fallidas no alteran el saldo de las cuentas.

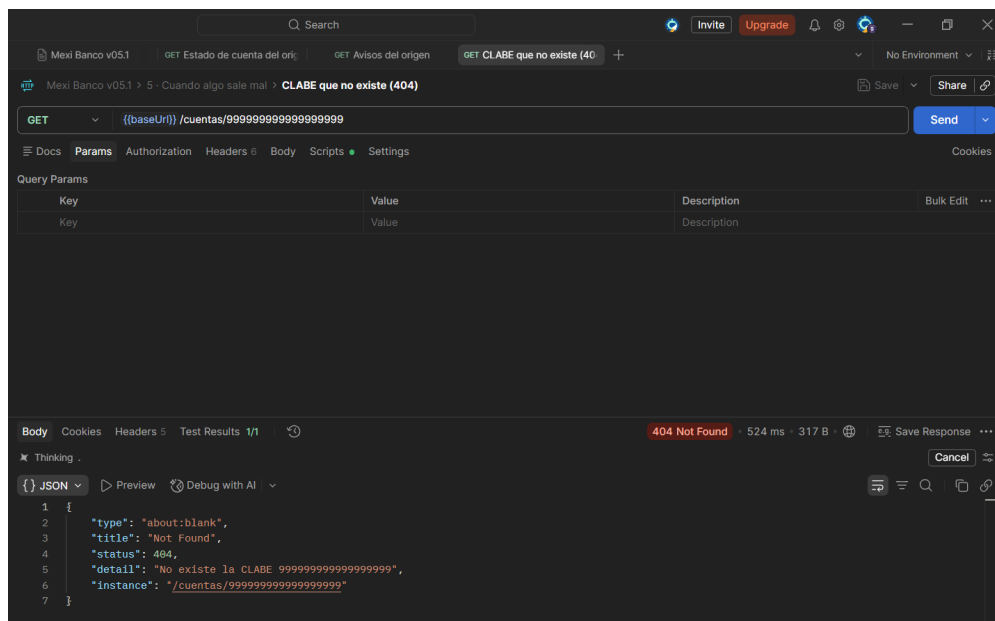


Figura 14: Error 404 al consultar una cuenta inexistente.

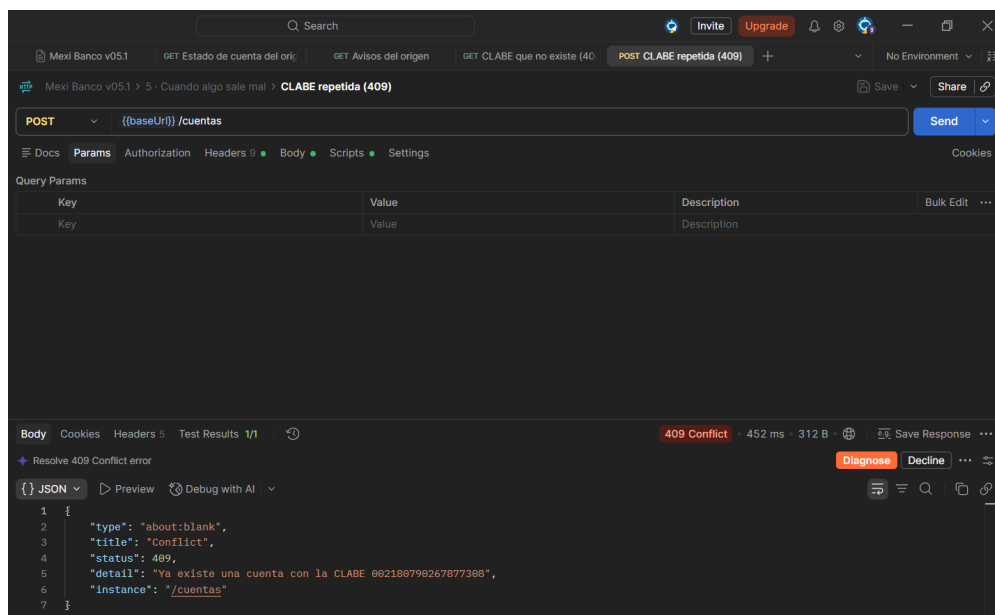


Figura 15: Error 409 por conflicto al intentar duplicar una CLABE.

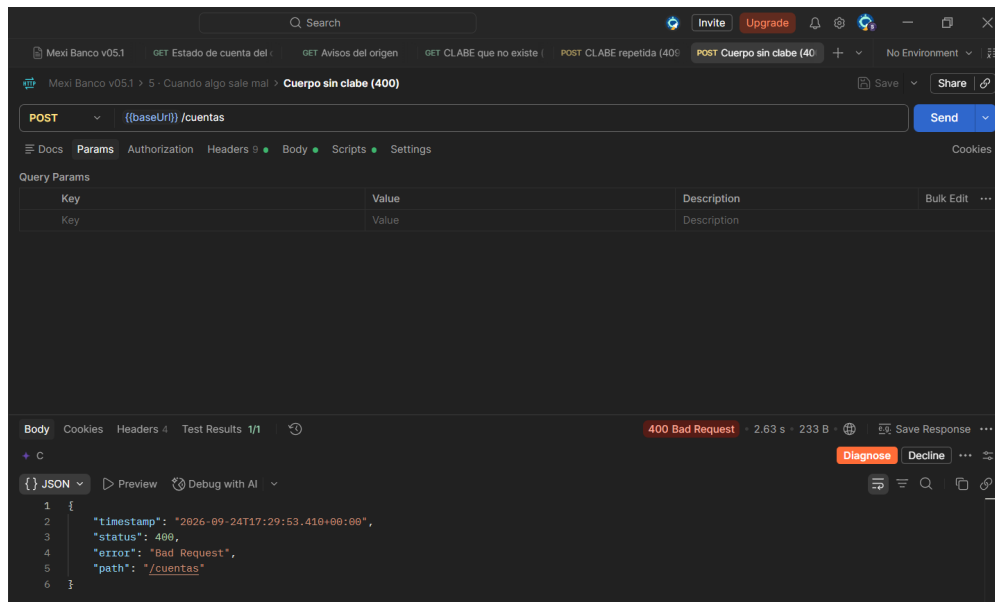


Figura 16: Error 400 por enviar un cuerpo de petición incompleto (sin CLABE)

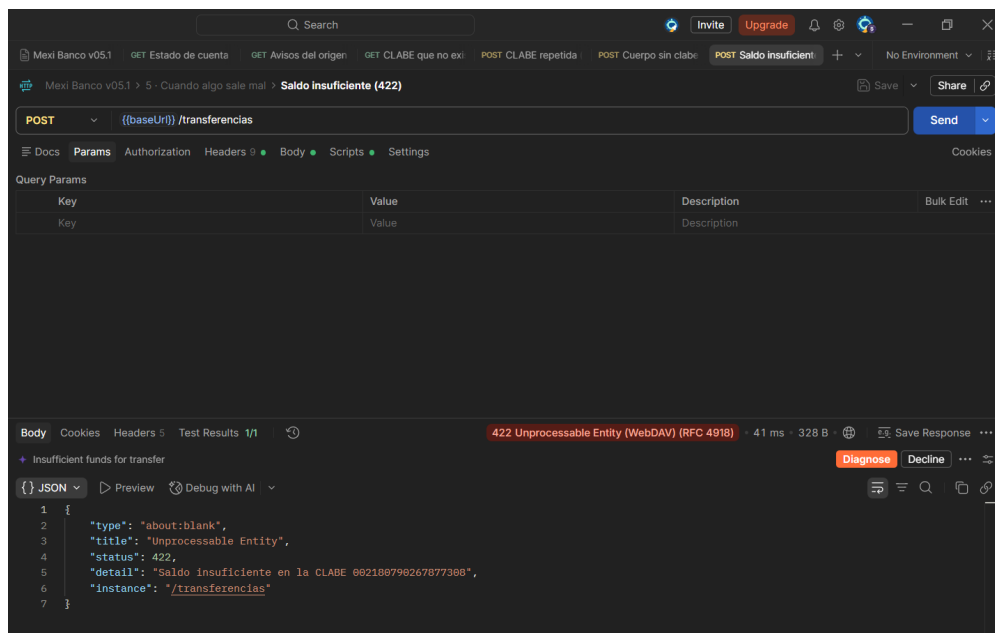


Figura 17: Error 422 al intentar transferir sin fondos suficientes.

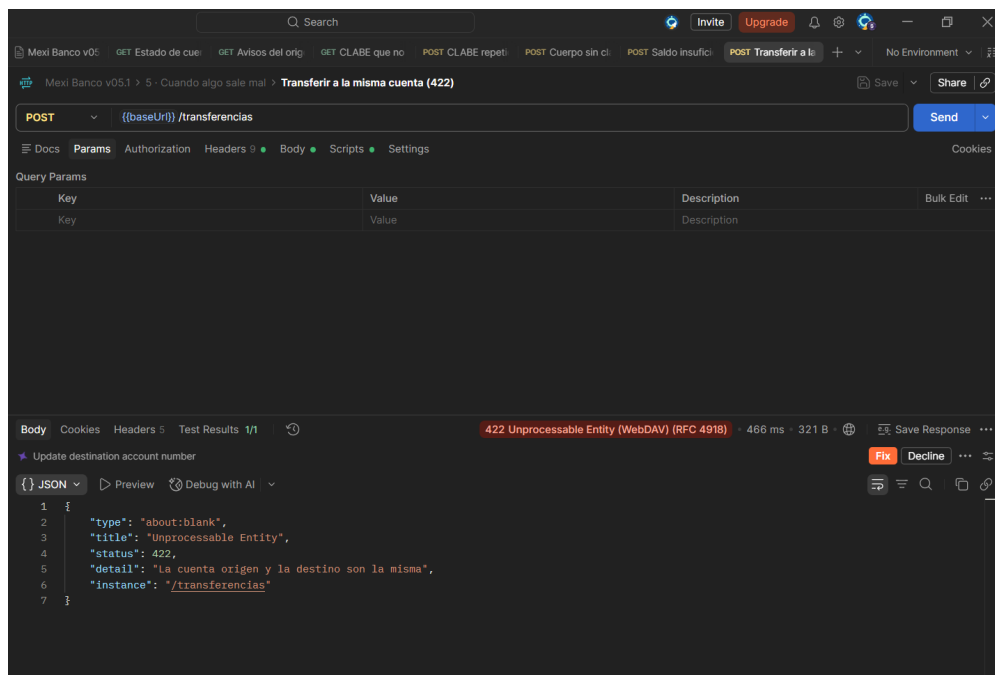


Figura 18: Error 422 al intentar realizar una transferencia a la misma cuenta origen.

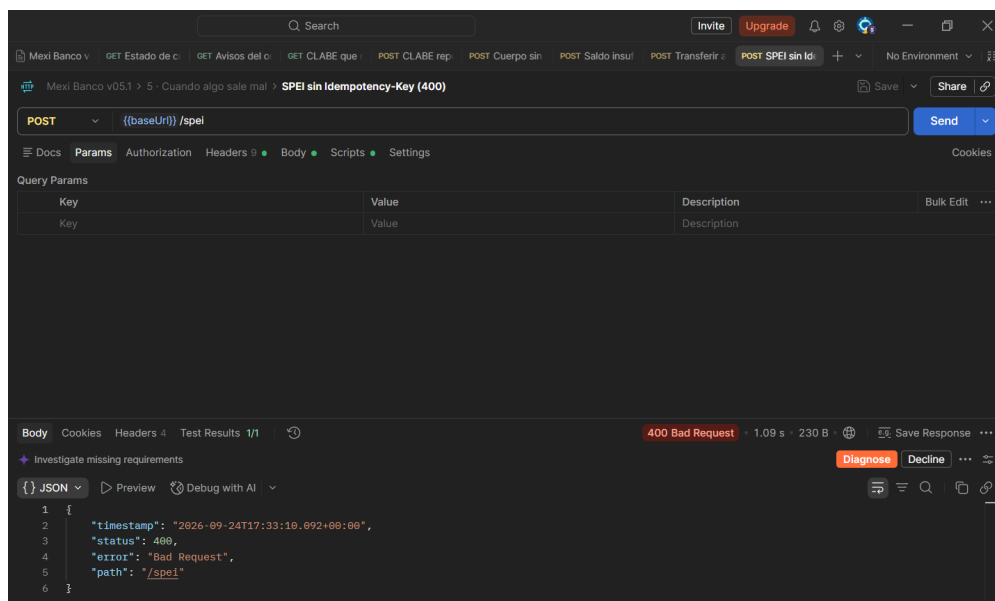


Figura 19: Error 400 por omitir el Idempotency-Key en un SPEI.

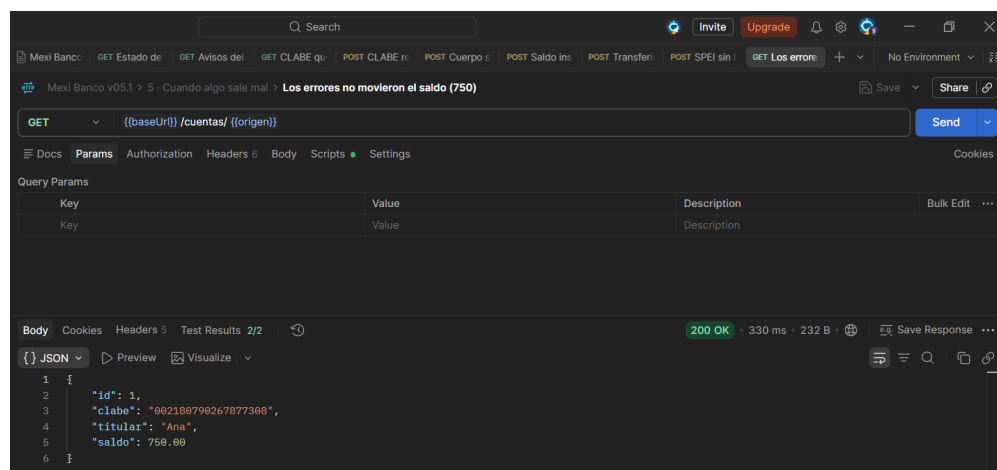


Figura 20: Consulta final verificando que los errores anteriores no movieron el saldo.

7. Persistencia con Volúmenes en Docker

Finalmente se analiza el comportamiento de los datos ante el ciclo de vida de los contenedores tambien se demuestra que un reinicio estándar conserva la información gracias al volumen de Docker mientras que destruir dicho volumen elimina permanentemente la base de datos local.

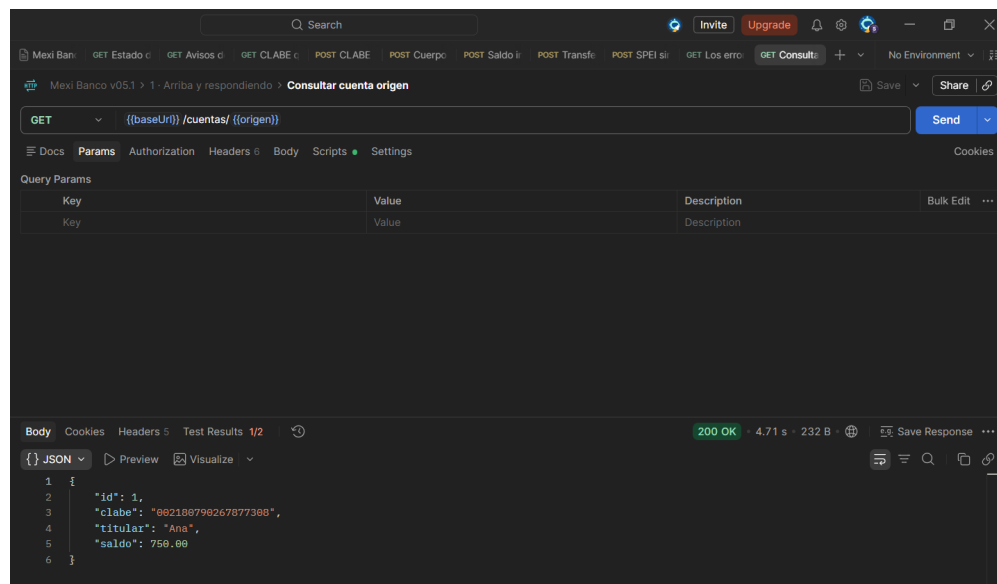


Figura 21: Tras ejecutar `docker compose down` y `up -d`, el saldo de \$750 persiste.

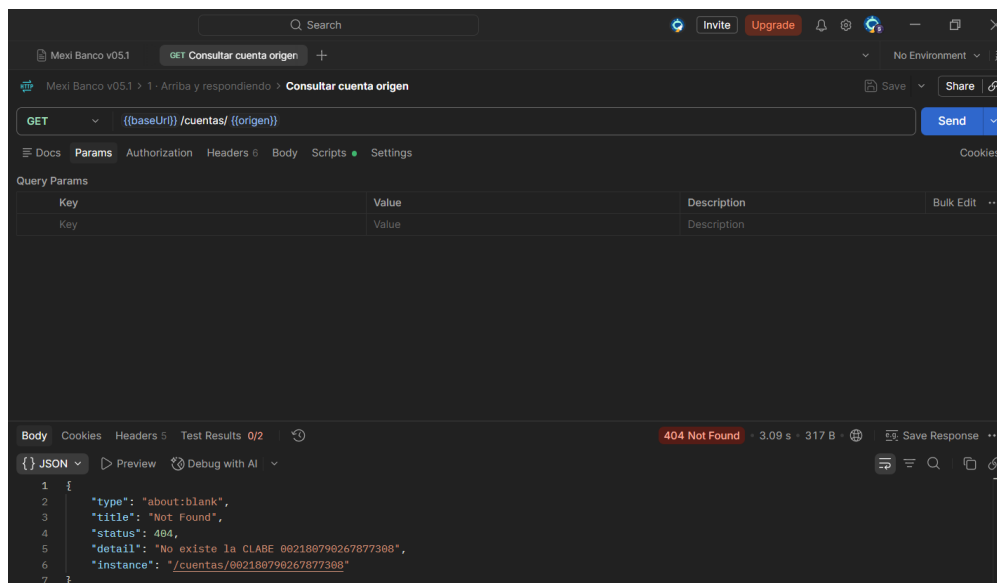


Figura 22: Tras ejecutar `docker compose down -v`, el volumen se destruye y la cuenta arroja 404.